SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA0613

NAME: Sree Laksha S V

REGNO:192421108

CO2-AT3

Q35. Debugging Closest Pair Code (Comparison with Wrong Variable)

Aim

To debug the given brute-force Closest Pair algorithm, identify the logical error in the comparison statement, correct the implementation, optimize the code by reducing unnecessary tuple indexing, analyze the time complexity, and provide the final commented program.

---

1. Introduction

The **Closest Pair Problem** is a computational geometry problem in which we are given a set of points on a two-dimensional plane. The objective is to determine the pair of points having the **minimum Euclidean distance** between them.

The brute-force approach compares every possible pair of points and keeps track of the smallest distance found.

---

2. Given Incorrect Python Program

```
import math

def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            x1, y1 = points[i]
            x2, y2 = points[j]
            dist = math.sqrt((x1-x2)**2 + (y1-y2)**2)
```

```
        if dist < i:      # ERROR
            min_dist = dist
    return min_dist
```

---

3. Error Analysis

The major logical error is:

if dist < i:

This is wrong because

- **dist** stores the Euclidean distance between two points.
- **i** is simply the loop index (0,1,2,3,...).

The algorithm should compare the newly calculated distance with the **smallest distance found so far**, not with the loop variable.

For example, Suppose

Distance = 4.2, Loop index i = 3

Comparison becomes

4.2 < 3

which is False.

Even if

Minimum distance = 10

the algorithm will never update it because it is comparing with **3 instead of 10**.

As a result, the smallest distance is not stored correctly.

---

4. Correct Logic

The comparison should be

if dist < min_dist:

    min_dist = dist

Now,

- Initially

min_dist = infinity

- First calculated distance is always smaller than infinity.
- Every new distance is compared with the smallest distance found so far.
- If smaller, update the minimum distance.

This guarantees the correct answer.

---

5. Optimized Python Program

daa.py - C:/Users/HP/AppData/Local/Programs/Python/Python311/daa.py (3.11.8)

File Edit Format Run Options Window Help

```
import math

def closest_pair(points):

    # Initialize minimum distance with infinity
    min_dist = float('inf')

    # Store total number of points
    n = len(points)

    # Compare every pair of points
    for i in range(n):

        # Store current point once
        x1, y1 = points[i]

        for j in range(i + 1, n):

            # Store second point once
            x2, y2 = points[j]

            # Calculate Euclidean distance
            dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)

            # Correct comparison
            if dist < min_dist:
                min_dist = dist

    return min_dist


# Driver Code
points = [(2,3), (12,30), (40,50), (5,1), (12,10), (3,4)]

answer = closest_pair(points)

print("Minimum Distance =", answer)
```

6. Sample Output

Minimum Distance = 1.4142135623730951

The closest points are

(2,3)

(3,4)

Distance

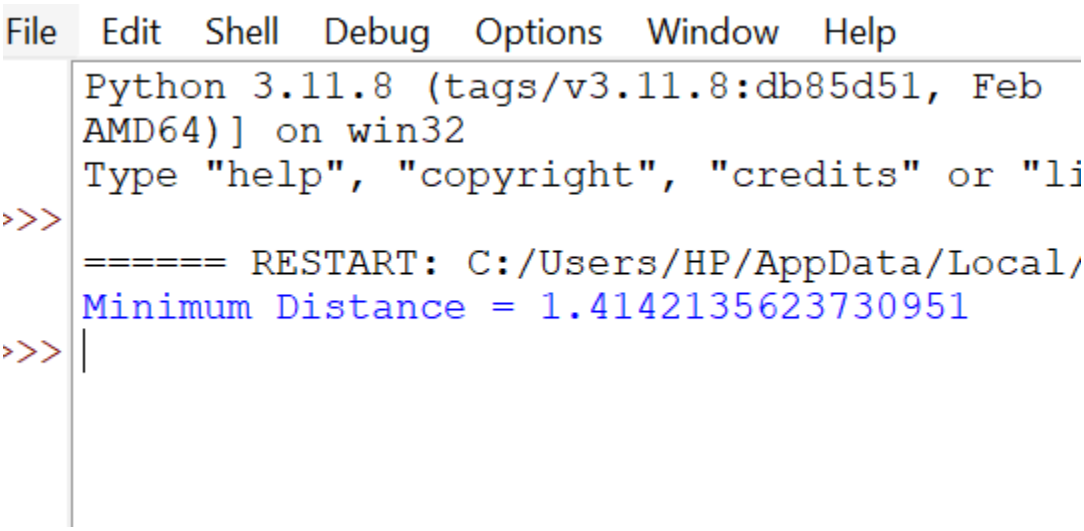
$\sqrt{((3-2)^2+(4-3)^2)}$

$= \sqrt{(1+1)}$

$= \sqrt{2}$

$= 1.414$

Output Screenshot:



---

9. Optimization Performed

The original code repeatedly accessed:

points[i][0]

points[i][1]

points[j][0]

points[j][1]

inside the nested loops.

Instead, we store the coordinates once:

x1, y1 = points[i]

x2, y2 = points[j]

Similarly, in C:

int x1 = points[i][0];

int y1 = points[i][1];

int x2 = points[j][0];

int y2 = points[j][1];

Benefits

- Fewer array accesses.
- Cleaner and more readable code.
- Slightly improved efficiency.
- Easier debugging.

---

10. Time Complexity Analysis

Let **n** be the number of points.

The outer loop runs:

n times

The inner loop runs approximately:

n-1

n-2

...

1

Total comparisons:

$n(n-1)/2$

Therefore,

Time Complexity:

$O(n^2)$

because every pair of points is examined exactly once.

Space Complexity:

$O(1)$

Only a few variables are used regardless of the number of points.

---

11. Debugging Summary

Issue	Incorrect Code	Correct Code
Comparison	if dist < i	if dist < min_dist
Problem	Compared distance with loop index	Compared with smallest distance found so far
Result	Minimum distance not updated correctly	Correct minimum distance maintained

---

12. Conclusion

The original program contained a logical error where the computed distance was compared against the loop variable i instead of the current minimum distance. This prevented the algorithm from preserving the smallest distance correctly. After replacing the condition with if (dist < min_dist) and optimizing coordinate access by storing point values in local variables, the brute-force algorithm correctly computes the closest pair. The corrected implementation examines every unique pair of points, requires **$O(n^2)$** time and **$O(1)$** extra space, and produces the correct minimum Euclidean distance.